

RAPPORT DE PROJET

Plateforme Big Data de Collecte et
Analyse d'Articles de Presse

Module	Architecture des Données / Big Data
Réalisé par	Younes Zouane
Filière	IADATA
Année universitaire	2025 / 2026

Table des Matières

1.	Introduction
2.	Contexte du projet
3.	Objectifs du projet
4.	Architecture globale
5.	Technologies utilisées
6.	Web Scraping
7.	Ingestion des données
8.	Data Lake
9.	Architecture Médaillon
10.	Transformation des données
11.	Streaming avec Kafka
12.	Data Warehouse
13.	Orchestration avec Airflow
14.	Visualisation des données
15.	Qualité des données
16.	Gouvernance des données
17.	Dockerisation
18.	Résultats obtenus
19.	Difficultés rencontrées
20.	Conclusion
21.	Perspectives futures
22.	Annexes

1. Introduction

Les médias numériques produisent quotidiennement une quantité massive d'informations sous forme d'articles de presse. Ces données représentent une source importante d'informations exploitables pour l'analyse des tendances médiatiques, l'identification des sujets dominants et la compréhension de l'actualité mondiale.

Dans ce projet, nous avons développé une plateforme Big Data capable de collecter automatiquement des articles depuis plusieurs sites d'actualité, puis de stocker, nettoyer, transformer et analyser ces données à travers une architecture distribuée moderne.

Le projet s'appuie sur plusieurs concepts importants du Big Data :

- Web Scraping — collecte automatique de données web
- Data Lake — stockage des données brutes à grande échelle
- Architecture Médaillon (Bronze / Silver / Gold)
- ETL / ELT — transformation des données
- Streaming de données avec Apache Kafka
- Data Warehouse — entreposage analytique
- Visualisation analytique avec Streamlit
- Qualité et gouvernance des données

2. Contexte du Projet

Les plateformes d'actualités génèrent continuellement de nouvelles informations. Afin d'exploiter ces données à grande échelle, il est nécessaire de mettre en place une architecture capable de répondre aux besoins suivants :

- Collecter automatiquement les données depuis des sources hétérogènes
- Traiter les données en mode batch et en streaming temps réel
- Conserver l'historique complet des données collectées
- Nettoyer, normaliser et transformer les données brutes
- Produire des indicateurs analytiques exploitables
- Visualiser les résultats sous forme de dashboards interactifs

Ce projet répond à ces besoins en mettant en oeuvre une plateforme complète de traitement de données intégrant les meilleures pratiques de l'ingénierie des données modernes.

3. Objectifs du Projet

3.1 Objectifs Fonctionnels

- Scraper automatiquement des articles de presse depuis Hespress et BBC News
- Stocker les données brutes dans un Data Lake structuré
- Construire une architecture Bronze / Silver / Gold
- Implémenter des traitements ETL avec Pandas
- Mettre en place un système de streaming avec Apache Kafka
- Produire des analyses décisionnelles exploitables
- Créer des tableaux de bord interactifs avec Streamlit

3.2 Objectifs Techniques

- Utiliser des technologies Big Data modernes et éprouvées
- Automatiser les pipelines de données avec Apache Airflow
- Garantir la qualité et la cohérence des données
- Dockeriser l'ensemble de l'architecture pour la portabilité

4. Architecture Globale

L'architecture générale du projet suit un flux de données linéaire et bien structuré, depuis la collecte jusqu'à la visualisation finale :

Étape	Composant	Description
1	Web Scraping	Collecte automatique des articles de presse
2	Kafka Streaming	Transmission des données sous forme d'événements
3	Bronze Layer	Stockage des données brutes sans transformation
4	Silver Layer	Nettoyage, normalisation et enrichissement
5	Gold Layer	Création des tables analytiques agrégées
6	Dashboard Analytics	Visualisation interactive des indicateurs

Web Scraping

Collecte automatique des articles de presse depuis Hespress et BBC News via les librairies requests et BeautifulSoup.

Kafka Streaming

Transmission des articles collectés sous forme de messages dans un topic Kafka, permettant un traitement temps réel.

Bronze Layer

Couche de stockage des données brutes issues du scraping, sans aucune transformation. Préserve l'intégralité des données originales.

Silver Layer

Couche de nettoyage et normalisation. Les données y sont dédoublées, nettoyées et enrichies (détection de langue).

Gold Layer

Couche analytique contenant les données agrégées prêtes pour la visualisation et l'analyse décisionnelle.

Dashboard

Interface Streamlit affichant les indicateurs clés sous forme de graphiques interactifs.

5. Technologies Utilisées

Technologie	Version	Rôle dans le projet
Python	3.10+	Langage de développement principal
BeautifulSoup4	4.x	Parsing HTML pour le Web Scraping
Pandas	2.x	Manipulation et transformation des données
Apache Kafka	3.x	Streaming et ingestion temps réel
Docker / Compose	Latest	Conteneurisation de l'architecture
Streamlit	1.x	Développement du dashboard analytique
Apache Airflow	2.x	Orchestration et planification des pipelines
JSON / CSV	—	Formats de stockage des données
requests	2.x	Requêtes HTTP pour le scraping

6. Web Scraping

6.1 Description

Le web scraping consiste à récupérer automatiquement les informations présentes sur les pages HTML des sites d'actualité. Cette étape constitue le point d'entrée de toute la chaîne de traitement de données.

6.2 Sites Utilisés

- Hespress — site d'actualité marocain en langue arabe et française
- BBC News — site d'actualité international en langue anglaise

6.3 Données Collectées

Pour chaque article scraped, les informations suivantes sont extraites et stockées :

Champ	Type	Description
titre	str	Titre principal de l'article
auteur	str	Nom de l'auteur ou journaliste
date_publication	datetime	Date et heure de publication
categorie	str	Rubrique ou section thématique
contenu	str	Corps complet du texte de l'article
source	str	Site d'origine (hespress / bbc)
url	str	URL complète de l'article

6.4 Fonctionnement

Le scraper suit un processus en 5 étapes :

- 1. Ouverture de la page principale du site cible
- 2. Identification et extraction des liens vers les articles
- 3. Accès à chaque page article individuellement
- 4. Extraction des champs définis via les sélecteurs CSS/HTML
- 5. Sauvegarde des données en formats JSON et CSV dans le Bronze Layer

7. Ingestion des Données

Le projet implémente deux modes complémentaires d'ingestion des données :

7.1 Batch Ingestion

Les articles sont collectés périodiquement via le scraper Python. Le pipeline batch est planifié et orchestré par Apache Airflow selon une fréquence configurable. Toutes les données collectées sont stockées dans le Bronze Layer.

7.2 Streaming Ingestion

Les articles nouvellement collectés sont également envoyés dans un topic Apache Kafka (news-topic) sous forme de messages JSON. Chaque article représente un événement indépendant dans le flux. Ce mode permet un traitement temps réel des nouvelles données dès leur collecte.

8. Data Lake

Le Data Lake constitue le référentiel central de stockage de toutes les données du projet, dans leurs différents états de traitement.

8.1 Description

Dans ce projet, le Data Lake est implémenté sous forme de système de fichiers structuré. Les données sont sauvegardées en formats JSON (données brutes) et CSV (données transformées), organisées selon l'architecture Médaille.

8.2 Structure des Dossiers

Répertoire	Contenu	Format
/data/bronze/	Données brutes issues du scraping	JSON, CSV
/data/silver/	Données nettoyées et normalisées	CSV
/data/gold/	Tables analytiques agrégées	CSV

9. Architecture Médaille

L'architecture Médaille est un pattern de conception de Data Lake qui organise les données en trois couches progressives de qualité croissante.

9.1 Bronze Layer — Données Brutes

La couche Bronze contient les données exactement telles qu'elles ont été collectées par le scraper, sans aucune modification. Elle preserve l'intégralité des données originales et sert de source de vérité historique.

- Fichier principal : all_articles.json
- Fichier secondaire : all_articles.csv
- Aucune transformation appliquée — données brutes intactes

9.2 Silver Layer — Données Nettoyées

La couche Silver contient les données après application des transformations de qualité. C'est la couche de référence pour les analyses.

- Suppression des doublons (même URL ou même titre)
- Nettoyage des espaces et caractères spéciaux
- Suppression des articles avec contenu vide ou invalide
- Détection automatique de la langue (langdetect)
- Normalisation des formats de date

9.3 Gold Layer — Données Analytiques

La couche Gold contient les données agrégées et prêtes pour la consommation analytique et la visualisation.

- Nombre d'articles par source et par période
- Répartition des articles par langue détectée
- Tendances médiatiques et sujets dominants
- Statistiques de qualité des données collectées

10. Transformation des Données

Les transformations ETL (Extract, Transform, Load) sont réalisées avec la librairie Pandas.

10.1 Nettoyage

- Suppression des lignes entièrement vides
- Normalisation des textes (suppression espaces multiples)
- Nettoyage des caractères spéciaux et encodages incorrects
- Validation des longueurs minimales de contenu

10.2 Enrichissement

- Détection automatique de la langue (langdetect)
- Extraction de métadonnées temporelles (jour, mois, année)

10.3 Agrégation

- Comptage des articles par source
- Regroupement chronologique par date

- Calcul des statistiques de distribution

11. Streaming avec Kafka

Apache Kafka est utilisé pour implémenter la couche de streaming temps réel du pipeline de données.

11.1 Description

Kafka permet de découpler la production et la consommation des données. Le scraper produit des messages, le consumer les traite indépendamment, permettant une scalabilité horizontale.

11.2 Configuration

Paramètre	Valeur
Topic	news-topic
Bootstrap Server	localhost:9092
Format des messages	JSON sérialisé
Groupe de consommateurs	news-consumers

11.3 Producer

Le producer Kafka lit chaque article collecté par le scraper et l'envoie dans le topic news-topic sous forme de message JSON sérialisé. Un message est produit par article scraped.

11.4 Consumer

Le consumer Kafka s'abonne au topic news-topic et reçoit les messages en temps réel. Il déserialise le JSON, affiche les données et peut les transmettre à d'autres composants du pipeline.

11.5 Avantages

- Traitement temps réel des nouvelles données
- Découplage producteur / consommateur
- Scalabilité horizontale par ajout de partitions
- Tolérance aux pannes et persistance des messages

12. Data Warehouse

Le projet prépare les données analytiques de la couche Gold destinées à alimenter un entrepôt de données décisionnel.

Table analytique	Description	Métriques
articles_par_source	Distribution par origine	Count, Pourcentage
articles_par_langue	Distribution par langue détectée	Count, Pourcentage
articles_par_date	Évolution temporelle	Count par jour/mois
tendances_sujets	Fréquence des mots-clés	TF-IDF, Count

Ces données peuvent être intégrées dans PostgreSQL, Amazon Redshift, ou tout autre SGBD analytique pour des analyses SQL avancées.

13. Orchestration avec Apache Airflow

Apache Airflow orchestre et planifie automatiquement l'exécution du pipeline de données complet.

13.1 DAG Implémenté

Le DAG (Directed Acyclic Graph) principal définit la séquence d'exécution des tâches avec leurs dépendances :

- Tâche 1 — scraping : Collecte des articles depuis les sources web
- Tâche 2 — bronze_to_silver : Nettoyage et normalisation des données
- Tâche 3 — silver_to_gold : Agrégation et création des tables analytiques

13.2 Avantages

- Automatisation complète du pipeline sans intervention manuelle
- Monitoring en temps réel de l'état des tâches
- Planification flexible (horaire, quotidienne, hebdomadaire)
- Gestion automatique des dépendances et relances en cas d'échec
- Interface web pour la visualisation des DAGs

14. Visualisation des Données

Le dashboard analytique a été développé avec Streamlit, un framework Python permettant de créer des applications web interactives.

14.1 Indicateurs Affichés

- Nombre total d'articles collectés par source
- Répartition des articles par langue détectée
- Évolution temporelle du volume de collecte
- Tendances analytiques des sujets médiatiques

14.2 Fonctionnement

Les données de la couche Gold sont chargées dans l'application Streamlit via Pandas. Elles sont ensuite affichées sous forme de graphiques interactifs (bar charts, pie charts, line charts) permettant une exploration visuelle des données.

15. Qualité des Données

Des contrôles de qualité systématiques ont été implémentés à chaque étape du pipeline.

Dimension	Vérification	Action corrective
Complétude	Titres et contenus manquants	Suppression de l'article
Validité	Longueur minimale du contenu	Filtrage des articles trop courts
Unicité	Doublons (URL ou titre)	Déduplication automatique
Cohérence	Format des dates	Normalisation du format
Exactitude	Langue détectée	Enrichissement automatique

16. Gouvernance des Données

La gouvernance des données assure la traçabilité, la transparence et la documentation de toutes les données du projet.

16.1 Éléments Mis en Place

- Structure de dossiers claire et documentée (bronze/silver/gold)
- Fichier README documentant l'architecture et les étapes
- Organisation modulaire des pipelines de traitement
- Séparation stricte des couches Bronze / Silver / Gold
- Journalisation des opérations de transformation

17. Dockerisation

Docker permet de déployer et reproduire facilement l'architecture du projet dans n'importe quel environnement.

17.1 Services Conteneurisés

Service	Image Docker	Rôle
Zookeeper	confluentinc/cp-zookeeper	Coordination du cluster Kafka
Kafka Broker	confluentinc/cp-kafka	Broker de messages Kafka

17.2 Avantages

- Portabilité : fonctionne identiquement sur tout système
- Isolation : chaque service dans son propre conteneur
- Facilité de déploiement avec docker-compose up
- Reproductibilité de l'environnement de développement

18. Résultats Obtenus

Le projet a permis de construire et valider un pipeline Big Data complet et fonctionnel.

18.1 Fonctionnalités Livrées

- Collecte automatique d'articles depuis Hespress et BBC News
- Pipeline ETL complet Bronze -> Silver -> Gold
- Streaming Kafka fonctionnel entre producer et consumer
- Dashboard Streamlit avec graphiques interactifs
- Orchestration Airflow avec DAG de 3 tâches
- Architecture dockerisée déployable en une commande

18.2 Indicateurs Analytiques Produits

- Répartition des articles par source (Hespress vs BBC)
- Distribution des langues détectées dans le corpus
- Évolution temporelle du volume de collecte
- Statistiques de qualité des données collectées

19. Difficultés Rencontrées

19.1 Compatibilité des Librairies

Certaines dépendances Python nécessitaient des versions spécifiques pour fonctionner ensemble. La gestion des conflits de versions via un fichier `requirements.txt` a été nécessaire.

19.2 Parsing HTML

Les structures HTML diffèrent significativement selon les sites web. Hespress utilise une structure différente de BBC News, nécessitant des sélecteurs CSS spécifiques pour chaque site.

19.3 Docker sur Windows

Le démarrage du moteur Docker Desktop sur Windows nécessitait l'activation de WSL2 (Windows Subsystem for Linux). La configuration correcte du réseau Docker était également nécessaire pour la communication entre conteneurs.

19.4 Configuration Kafka

La communication entre le producer et le consumer Kafka demandait une configuration précise des paramètres réseau (`advertised.listeners`, `bootstrap.servers`) pour fonctionner correctement dans un environnement Docker.

20. Conclusion

Ce projet a permis de mettre en oeuvre une architecture Big Data complète et fonctionnelle, capable de collecter, transformer et analyser des données issues de sites d'actualité en temps réel.

Les différentes technologies utilisées (Python, Kafka, Docker, Streamlit, Airflow) ont permis de construire un pipeline moderne intégrant ingestion batch, streaming temps réel, transformation ETL et visualisation analytique.

Le projet répond aux objectifs définis dans le cahier des charges et constitue une introduction pratique et concrète aux architectures Big Data distribuées, couvrant l'ensemble du cycle de vie de la donnée depuis sa collecte jusqu'à sa visualisation.

Les compétences acquises lors de ce projet — scraping, ETL, streaming, orchestration, conteneurisation — sont directement applicables dans un contexte professionnel d'ingénierie des données.

21. Perspectives Futures

Plusieurs améliorations et extensions peuvent être envisagées pour enrichir la plateforme :

21.1 Extensions Techniques

- Intégration d'Apache Spark pour le traitement distribué à grande échelle
- Déploiement sur le cloud (AWS, GCP, Azure) avec services managés
- Ajout de davantage de sources de presse nationales et internationales
- Implémentation d'alertes temps réel sur événements importants

21.2 Extensions Analytiques

- Analyse NLP avancée : résumé automatique, extraction d'entités
- Détection automatique de fake news par modèles de machine learning
- Analyse de sentiment sur les articles collectés
- Utilisation de modèles LLM pour la classification thématique

22. Annexes

22.1 Structure Finale du Projet

```
news-bigdata-project/  
  ■■■ scraper/  
    ■ ■■■ scraper.py  
    ■ ■■■ sites/ (hespress.py, bbc.py)  
    ■■■ spark/  
      ■ ■■■ bronze_to_silver.py  
      ■ ■■■ silver_to_gold.py  
      ■■■ streaming/  
        ■ ■■■ producer.py  
        ■ ■■■ consumer.py  
      ■■■ airflow/dags/  
        ■ ■■■ pipeline_dag.py  
      ■■■ dashboards/  
        ■ ■■■ app.py  
      ■■■ data/ (bronze/ silver/ gold/)  
      ■■■ docker-compose.yml  
      ■■■ README.md
```

22.2 Captures d'Écran à Ajouter

- 1. Structure du projet dans l'explorateur de fichiers
- 2. Exécution du scraper avec logs de collecte
- 3. Fichiers Bronze générés (all_articles.json)
- 4. Exécution Silver Layer — logs de nettoyage
- 5. Exécution Gold Layer — statistiques produites
- 6. Docker containers en cours d'exécution
- 7. Kafka producer / consumer — échange de messages

- 8. Dashboard Streamlit — graphiques analytiques
- 9. DAG Airflow — vue du pipeline et statut des tâches

22.3 Bibliographie

- Documentation officielle Python 3 — <https://docs.python.org>
- Documentation BeautifulSoup4 — <https://www.crummy.com/software/BeautifulSoup/>
- Documentation Apache Kafka — <https://kafka.apache.org/documentation/>
- Documentation Docker — <https://docs.docker.com>
- Documentation Streamlit — <https://docs.streamlit.io>
- Documentation Apache Airflow — <https://airflow.apache.org/docs/>
- Documentation Pandas — <https://pandas.pydata.org/docs/>